

then assigned to the parameters. For example, enter the following into the interactive shell:

```
❶ >>> def greeting(name, species):  
...     print(name + ' is a ' + species)  
...  
❷ >>> greeting('Zophie', 'cat')  
Zophie is a cat
```

In the `def` statement, `name` and `species` are parameters ❶. In the function call, `'Zophie'` and `'cat'` are arguments ❷. These two terms are often confused with each other. Remember that parameters and arguments are just other names for variables and values, respectively, when they are used in this context.

Type Coercion vs. Type Casting

You can convert an object of one type to an object of another type. For example, `int('42')` converts a string `'42'` to an integer `42`. In actuality, the string object `'42'` isn't converted so much as the `int()` function creates a new integer object based on the original object. When conversion is done *explicitly* like this, we're *casting* the object, although programmers often still refer to this process as converting the object.

Python will often implicitly do a type conversion, such as when evaluating the expression `2 + 3.0` to `5.0`. Values, such as the `2` and `3.0`, are coerced to a common data type that the operator can work with. This conversion, which is done *implicitly*, is called type *coercion*.

Coercion can sometimes lead to surprising results. The Boolean `True` and `False` values in Python can be coerced to the integer values `1` and `0`, respectively. Although you'd never write Booleans as those values in real-world code, this means that the expression `True + False + True` is the equivalent of `1 + 0 + 1` and evaluates to `2`. After learning this, you might think that passing a list of Booleans to `sum()` would be a good way to count the number of `True` values in a list. But it turns out that calling the `count()` list method is faster.

Properties vs. Attributes